UNIVERSIDAD PANAMERICANA

Facultad de ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas y TIC

Análisis y Diseño de Sistemas

Análisis y Diseño de Sistemas

(DERCAS y Propuesta de la Solución)

Kevin Eleazar Azurdia Chamalé (000137188)

Juan Diego Wosbely Sac López (000149360)

Chimaltenango, Chimaltenango, marzo de 2026

Tabla de contenido

Documento de Especificación de Requerimientos (DERCAS) - P2 2

Introducción: 2

Antecedentes 2

Objetivos 3

Objetivo General: 3

Objetivos Específicos: 3

Descripción de procesos 4

1. Proceso de registro de usuario 4
2. Proceso de inicio de sesión 4
3. Proceso de publicación de fotografía 4
4. Proceso de votación (like/dislike) 5
5. Proceso de comentarios 5
6. Proceso de búsqueda 6
7. Proceso de administrador 6

Diagrama de Bloques de Procesos 7

Descripción de Procedimientos (Arquitectura de Bloques) 7

Capa de Cliente (Frontend): 7

API Gateway: 8

Ecosistema de Microservicios (Backend): 8

Capa de persistencia almacenamiento: 8

Descripción de Procedimientos: 8

Diagramas de flujo de procesos 9

1. Diagrama de flujo, Registro de Usuarios 9
2. Diagrama de flujo, Inicio de Sesión 10
3. Diagrama de flujo, Publicación de fotografía 11
4. Diagrama de flujo, Proceso de Votación 12
5. Diagrama de flujo, Proceso de comentarios 13
6. Diagrama de flujo, Proceso de búsqueda 14
7. Diagrama de flujo, Proceso de administrador 15

Listado de Requerimientos Funcionales 16

Definición del Alcance 17

Identificación de módulos que integran la aplicación 18

Diagrama de componentes 18

Definición de requerimientos no funcionales 19

Diagramas de casos de uso 20

Diagramas de flujo de datos 21

Prototipos e interfaces de los requerimientos funcionales 22
Figura 1 y 2: Pantalla de Login y Registro 22
Figura 3: Feed Principal 23
Figura 4: Detalle de publicación y comentarios 24
Figura 5: Búsqueda / Explorar 25
Figura 6: Nueva Publicación 26
Figura 7: Panel de Administración 27
Recursos tecnológicos que se van a utilizar 28
Prioridad de los requerimientos a desarrollar 29
Prioridad alta (Semanas 5 a 8) 29
Prioridad Media (Semanas 9 a 13) 30
Prioridad Baja (Semanas 14 a 16) 30

Documento de Especificación de Requerimientos (DERCAS) - P2

Introducción:

Hipstagram es una plataforma de red social orientada a la publicación y descubrimiento de fotografías. El sistema permite a usuarios registrados publicar imágenes con descripciones de hasta 128 caracteres y hashtags, interactuar mediante votos (like/dislike) y comentarios paginados, y descubrir contenido a través de un motor de búsqueda por hashtag exacto y texto libre. La plataforma implementa una arquitectura de microservicios contenerizados con Docker, desplegada en infraestructura de nube AWS (EC2, RDS PostgreSQL, S3). La seguridad se gestiona mediante autenticación JWT con cifrado bcrypt, control de acceso basado en roles (RBAC) para los roles USER y ADMIN, y moderación automática de contenido. El sistema incluye un log de auditoría de transacciones accesible exclusivamente por el rol ADMIN, implementado mediante lista doblemente enlazada con persistencia en base de datos. El ciclo de vida del software contempla un pipeline CI/CD con Jenkins y SonarQube con Quality Gate obligatorio. Este documento presenta la especificación de requerimientos funcionales y no funcionales, los diagramas de procesos y flujos de datos, la identificación de módulos basados en microservicios, los prototipos de interfaz, el presupuesto tecnológico y la priorización del desarrollo del proyecto Hipstagram.

Antecedentes

La proliferación de plataformas de contenido visual como Instagram, Pinterest y Flickr ha demostrado que existe una demanda sostenida de sistemas capaces de gestionar publicación de imágenes a escala, con mecanismos robustos de interacción social, moderación de contenido y búsqueda etiquetada. Estas plataformas presentan desafíos técnicos complejos: autenticación segura, control de acceso por roles, almacenamiento distribuido de archivos multimedia, procesamiento concurrente de votos y auditoría de transacciones.

Hipstagram nace para resolver estos mismos desafíos aplicando estándares modernos de ingeniería de software: arquitectura de microservicios independientes (auth-service, post-service, vote-service, comment-service, search-service) que garantizan escalabilidad horizontal y desacoplamiento de componentes; contenedorización con Docker para asegurar portabilidad y consistencia entre entornos de desarrollo, pruebas y producción; autenticación mediante JWT firmado con bcrypt para el hash de contraseñas; y despliegue automatizado en AWS mediante un pipeline CI/CD con Jenkins y SonarQube. El proyecto aborda de forma integral los retos de los sistemas distribuidos modernos: seguridad, gestión de archivos en nube (S3), integridad referencial en base de datos relacional (PostgreSQL con RDS), auditoría de acciones y automatización del ciclo de vida del software.

Objetivos

Objetivo General:

Diseñar e implementar Hipstagram como una solución de red social basada en microservicios independientes (auth-service, post-service, vote-service, comment-service, search-service) desarrollados con Node.js/Express y TypeScript, con frontend en Angular, base de datos relacional PostgreSQL gestionada en AWS RDS, almacenamiento de imágenes en AWS S3, autenticación mediante JWT con cifrado bcrypt, control de acceso basado en roles (RBAC), moderación de contenido por microservicio, log de auditoría con lista doblemente enlazada, integridad referencial con RESTRICT en claves foráneas, y despliegue automatizado en AWS EC2 con Docker Compose mediante un pipeline CI/CD con Jenkins y SonarQube.

Objetivos Específicos:

1. Implementar el servicio de autenticación con registro de usuarios, inicio de sesión seguro mediante JWT firmado, hash de contraseñas con bcrypt, y control de acceso basado en roles (ADMIN y USER) aplicado mediante middleware en cada endpoint del backend.
2. Desarrollar el servicio de publicaciones con soporte para subida de imágenes a AWS S3, validación de descripción máxima de 128 caracteres, gestión de hashtags normalizados, y operaciones CRUD protegidas por integridad referencial con RESTRICT en claves foráneas de PostgreSQL.
3. Implementar el servicio de votos con control de duplicidad mediante restricción UNIQUE (id_usuario, id_post), lógica de cambio de reacción y cancelación de voto, todo dentro de transacciones PostgreSQL con COMMIT, ROLLBACK y SAVEPOINT.
4. Desarrollar el servicio de comentarios con validación de longitud, paginación de resultados y registro de operaciones en el log de auditoría implementado con lista doblemente enlazada en memoria con persistencia en base de datos.
5. Implementar el motor de búsqueda con soporte para consultas por hashtag exacto y texto libre sobre descripciones, con resultados ordenados por cantidad de likes de forma descendente.
6. Configurar el pipeline CI/CD con Jenkins para ejecución automática de lint, pruebas unitarias, análisis de calidad con SonarQube y build de imágenes Docker en cada Pull Request, con despliegue automatizado a AWS EC2 en merge a main.
7. Documentar los contratos de la API utilizando la especificación OpenAPI, definiendo los endpoints, parámetros, esquemas de request/response y códigos de estado de cada microservicio.
8. Desplegar la solución completa en AWS EC2 con Docker Compose, base de datos en RDS PostgreSQL, imágenes en S3 y configuración de SSH en puerto 443.

Descripción de procesos

A continuación, se describen los procesos fundamentales que componen la operación de Hipstagram, desde la interacción de usuarios hasta la administración del sistema.

1. Proceso de registro de usuario

El proceso de registro permite a una persona convertirse en usuario de Hipstagram. El flujo es el siguiente:

1.1. El usuario ingresa a la aplicación y selecciona la opción “Registrarse”.
1.2. El sistema solicita los datos necesarios: Nombre completo, correo electrónico y contraseña.
1.3. El usuario completa el formulario y envía la información.
1.4. El sistema valida que el correo no esté registrado previamente y que la contraseña cumpla con los requisitos de seguridad.
1.5. Si la validación es exitosa, el sistema crea la cuenta con rol USUARIO por defecto.
1.6. El sistema redirige al usuario a la página de inicio de sesión para que ingrese con sus nuevas credenciales.

2. Proceso de inicio de sesión

El proceso de autenticación permite a un usuario registrado acceder a la plataforma:

2.1. El usuario ingresa su correo electrónico y contraseña.
2.2. El sistema verifica que las credenciales sean correctas.
2.3. Si son válidas, el sistema otorga acceso y redirige al feed principal.
2.4. Si las credenciales son incorrectas, el sistema muestra un mensaje de error sin revelar si el error es en el correo o la contraseña (por seguridad).

3. Proceso de publicación de fotografía

Este proceso permite a un usuario compartir contenido visual con la comunidad:

3.1. El usuario autenticado accede a la opción “Subir foto”.
3.2. El sistema presenta un formulario con los siguientes campos.
- Selección de archivo de imagen.

- Descripción (máximo 128 caracteres, opcional).
- Hashtags (opcional, separados por espacio o coma).

3.3. El usuario selecciona una imagen, escribe una descripción y/o hashtags, y envía.

3.4. El sistema recibe la publicación y almacena los metadatos en la base de datos relacional.

3.5. El archivo de imagen se carga automáticamente en el repositorio de la nube.

3.6. La publicación se hace visible de forma inmediata en el feed principal y en los resultados de búsqueda.

4. Proceso de votación (like/dislike)

Los usuarios pueden expresar su reacción ante las publicaciones de otros:

4.1. El usuario autenticado visualiza una publicación en el feed o en el modo Explorar.

4.2. Debajo de la publicación, el usuario encuentra dos botones: “Like” y “Dislike”.

4.3. Al hacer clic en “Like”:

- Si el usuario no había votado antes, se registra un like y el contador aumenta.
- Si el usuario había votado “Dislike”, el voto se cambia a “Like” y los contadores se actualizan.

4.4. Al hacer clic en “Dislike”:

- Si el usuario no había votado antes, se registra un dislike y el contador aumenta.
- Si el usuario había votado “Like”, el voto se cambia a “Dislike” y los contadores se actualizan.

4.5. El usuario puede retirar su voto haciendo clic nuevamente en el voto activo.

5. Proceso de comentarios

Los usuarios pueden comentar las publicaciones:

5.1. El usuario autenticado selecciona una publicación para ver su detalle.

5.2. En la sección de comentarios, escribe su mensaje en un campo de texto.

5.3. El sistema valida que el comentario no esté vacío y que cumpla con las normas de contenido.

5.4. Al enviar, el comentario se publica inmediatamente y se muestra al final de la lista de comentarios.

5.5. Los comentarios se muestran paginados (ejemplo: 10 comentarios por página) para optimizar la visualización.

6. Proceso de búsqueda

Los usuarios pueden buscar publicaciones de dos formas:

6.1. Búsqueda por hashtag:

6.1.1. El usuario escribe un hashtag en el buscador (ejemplo: #atardecer).

6.1.2. El sistema muestra todas las publicaciones que contengan ese hashtag exacto.

6.1.3. Los resultados se ordenan por cantidad de likes (descendente) y se paginan.

6.2. Búsqueda por texto libre:

6.2.1. El usuario escribe una o varias palabras en el buscador.

6.2.2. El sistema busca coincidencias en las descripciones de las publicaciones.

6.2.3. Los resultados se ordenan por relevancia y/o cantidad de likes, y se paginan.

7. Proceso de administrador

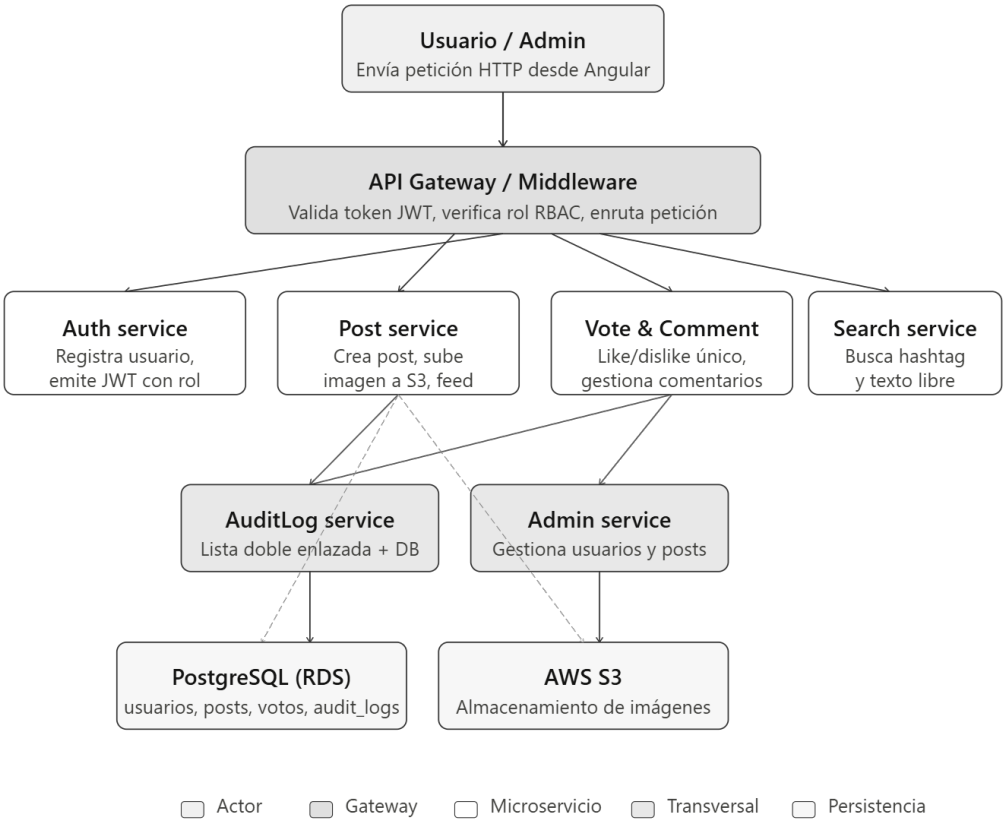
7.1. El ADMIN accede al panel de control mediante credenciales: rol de administrador.

7.2. Gestión de usuarios: Puede visualizar el listado de usuarios y desactivar cuentas que incumplan normas.

7.3. Gestión de contenido: Puede eliminar publicaciones de forma definitiva para mantener la integridad de la plataforma.

Diagrama de Bloques de Procesos

A continuación, se presenta el diagrama de bloques que define la arquitectura de alto nivel del sistema Hipstagram. Este modelo visualiza la separación estructural en capas lógicas, mostrando como interactúa la aplicación cliente con el ecosistema de microservicios a través del API Gateway, hasta llegar a la capa de persistencia de datos.



Descripción de Procedimientos (Arquitectura de Bloques)

A continuación, se describen las funciones y responsabilidades de cada componente integrado en la arquitectura del sistema:

Capa de Cliente (Frontend):

Representa la interfaz con la que interactúa el usuario final, desarrollada por entornos web y móviles. Su función es capturar las entradas del usuario y presentar de forma visual los datos obtenidos desde el servidor.

API Gateway:

Actúa como el punto único de entrada para todas las solicitudes provenientes del cliente. Se encarga de autenticar las peticiones y redirigirlas al microservicio correspondiente, optimizando la comunicación y seguridad del ecosistema.

Ecosistema de Microservicios (Backend):

- Servicio de autenticación: Encargado de la lógica del registro de nuevos usuarios y la validación de credenciales para el inicio de sesión.
- Servicio de contenido e interacciones: Gestiona la creación de nuevos posts, el almacenamiento de metadatos de las imágenes y el procesamiento de los contadores de "likes" y "dislikes".

Capa de persistencia almacenamiento:

- Bases de Datos Relacional: Almacena la información estructurada como perfiles de usuario, relaciones de votos.
- Almacenamiento de imágenes: Repositorio especializado para el guardado físico de los archivos multimedia en la nube, garantizando su disponibilidad y carga rápida.

Descripción de Procedimientos:

- Servicio de Búsqueda: Encargado de indexar y procesar las consultas por hashtags y texto libre, permitiendo la recuperación eficiente de publicaciones desde la base de datos relacional.

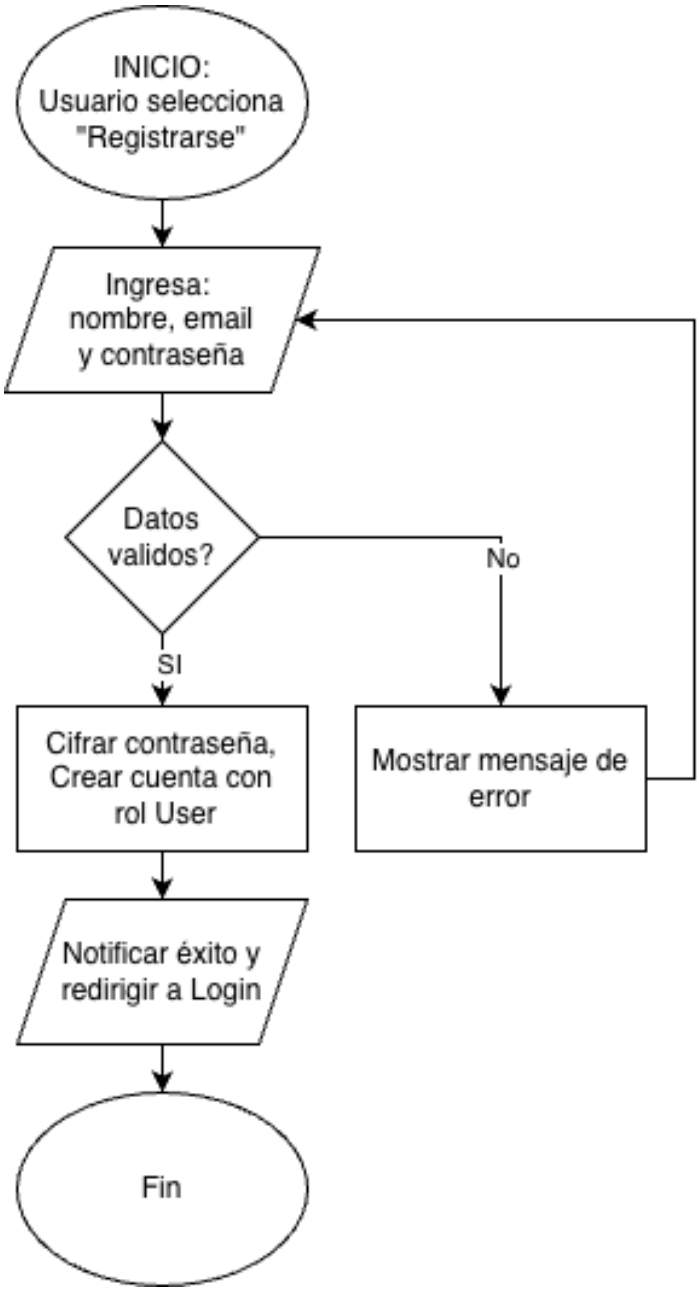
Se selecciona AWS como proveedor de infraestructura en la nube por razones técnicas y operativas concretas: (1) EC2 permite desplegar contenedores Docker directamente sin necesidad de un orquestador adicional, lo que simplifica la configuración del pipeline CI/CD con Jenkins self-hosted; (2) RDS con PostgreSQL gestiona automáticamente backups, parches y alta disponibilidad de la base de datos relacional, eliminando la administración manual del motor; (3) S3 provee almacenamiento de objetos con alta durabilidad (99.999999999%) y URLs firmadas para acceso controlado a las imágenes de los usuarios; (4) CloudWatch centraliza los logs estructurados de cada microservicio y permite configurar alertas sobre métricas de salud, lo que facilita la observabilidad del sistema en producción; (5) AWS Secret Manager permite gestionar variables de entorno y credenciales sin versionar secretos en el repositorio Git, cumpliendo con las buenas prácticas de seguridad del pipeline CI/CD.

Diagramas de flujo de procesos

En esta sección se detalla la lógica operativa de los procesos principales del sistema Hipstagram, utilizando la simbología estándar para representar decisiones, proceso y entradas de datos.

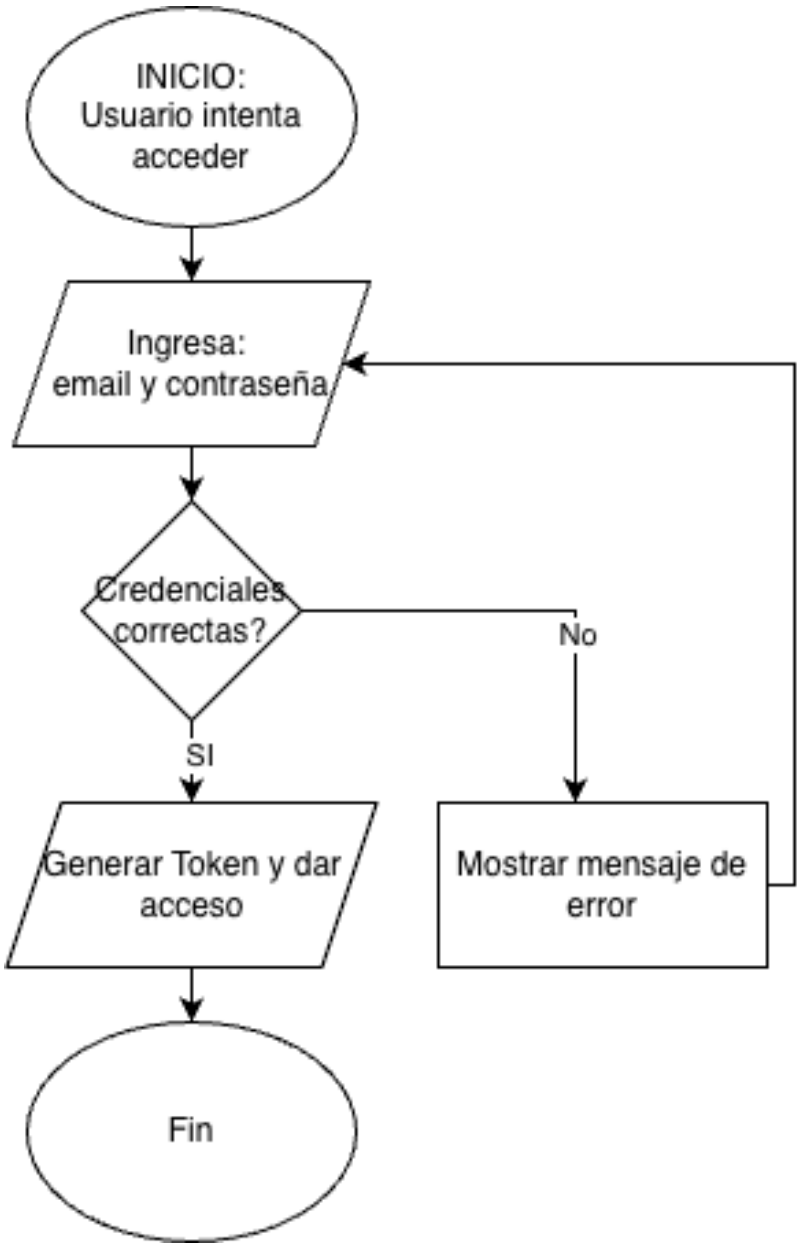
1. Diagrama de flujo, Registro de Usuarios

Este flujo describe los pasos desde que el usuario ingresa sus datos personales hasta que el sistema valida que el correo o nombre de usuario no existan previamente en la base de datos.



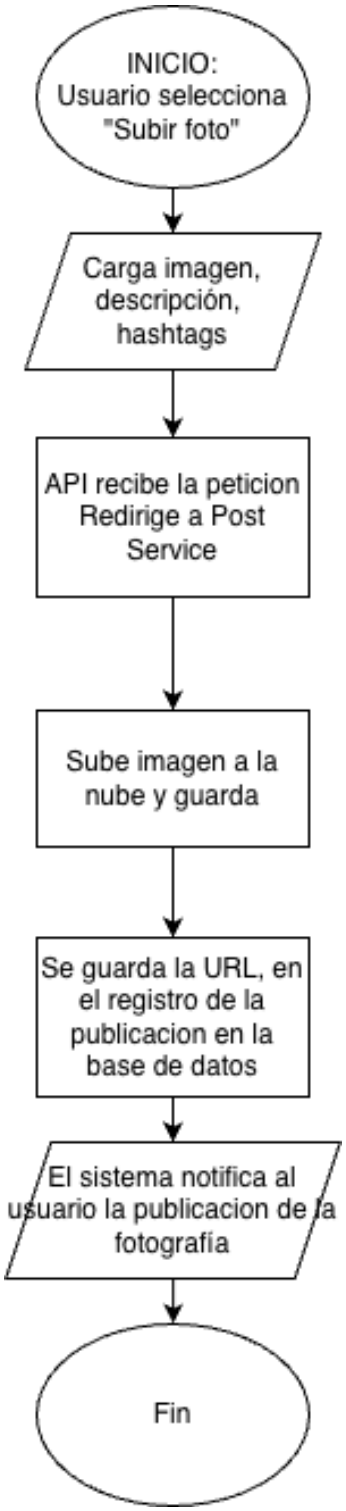
2. Diagrama de flujo, Inicio de Sesión

Describe la validación de credenciales y el manejo de errores en caso de contraseñas incorrectas, garantizando el acceso seguro al perfil del usuario.



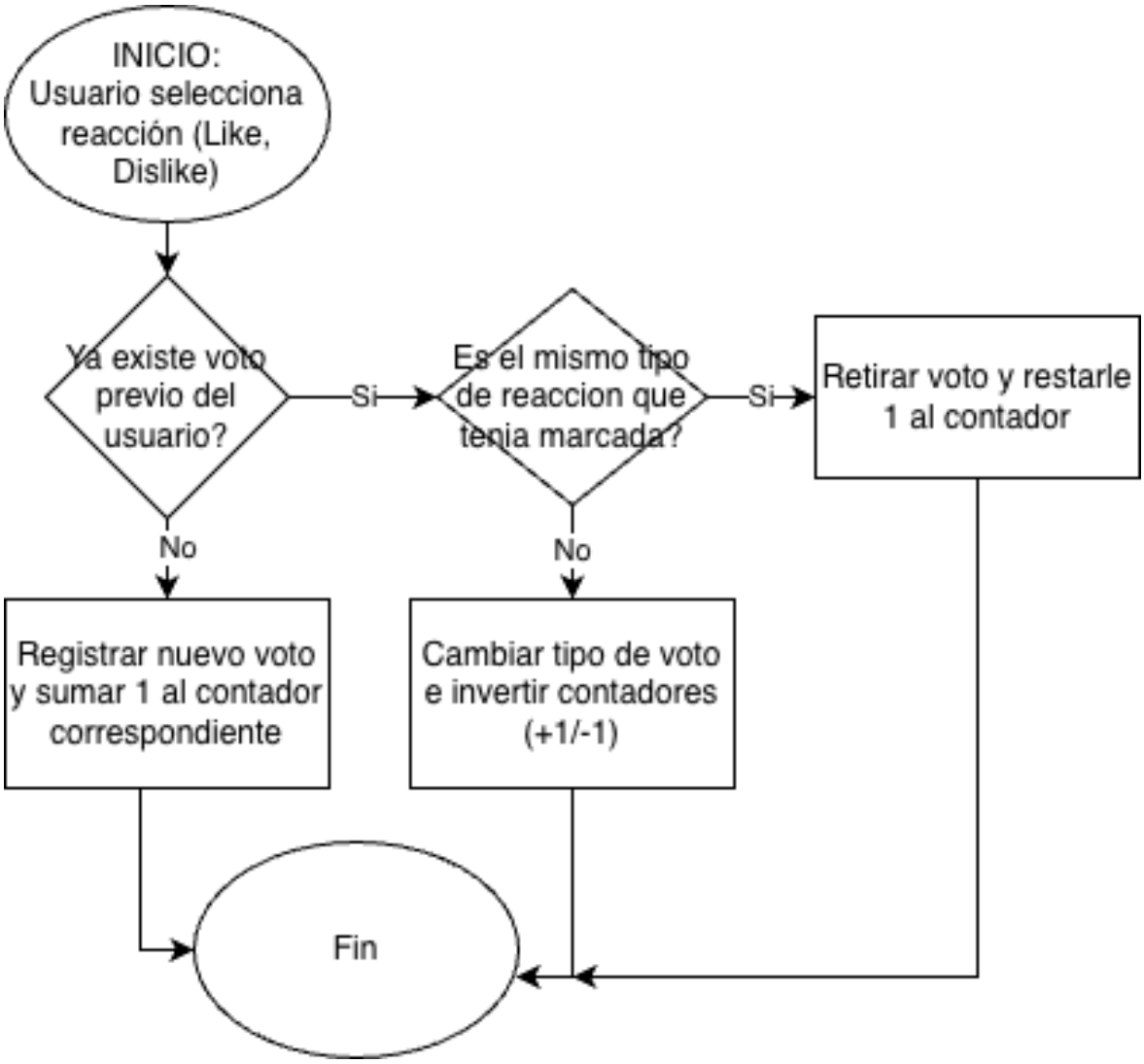
3. Diagrama de flujo, Publicación de fotografía

Este es el flujo más robusto del sistema. Muestra como una publicación es publicada en el feed del sistema.



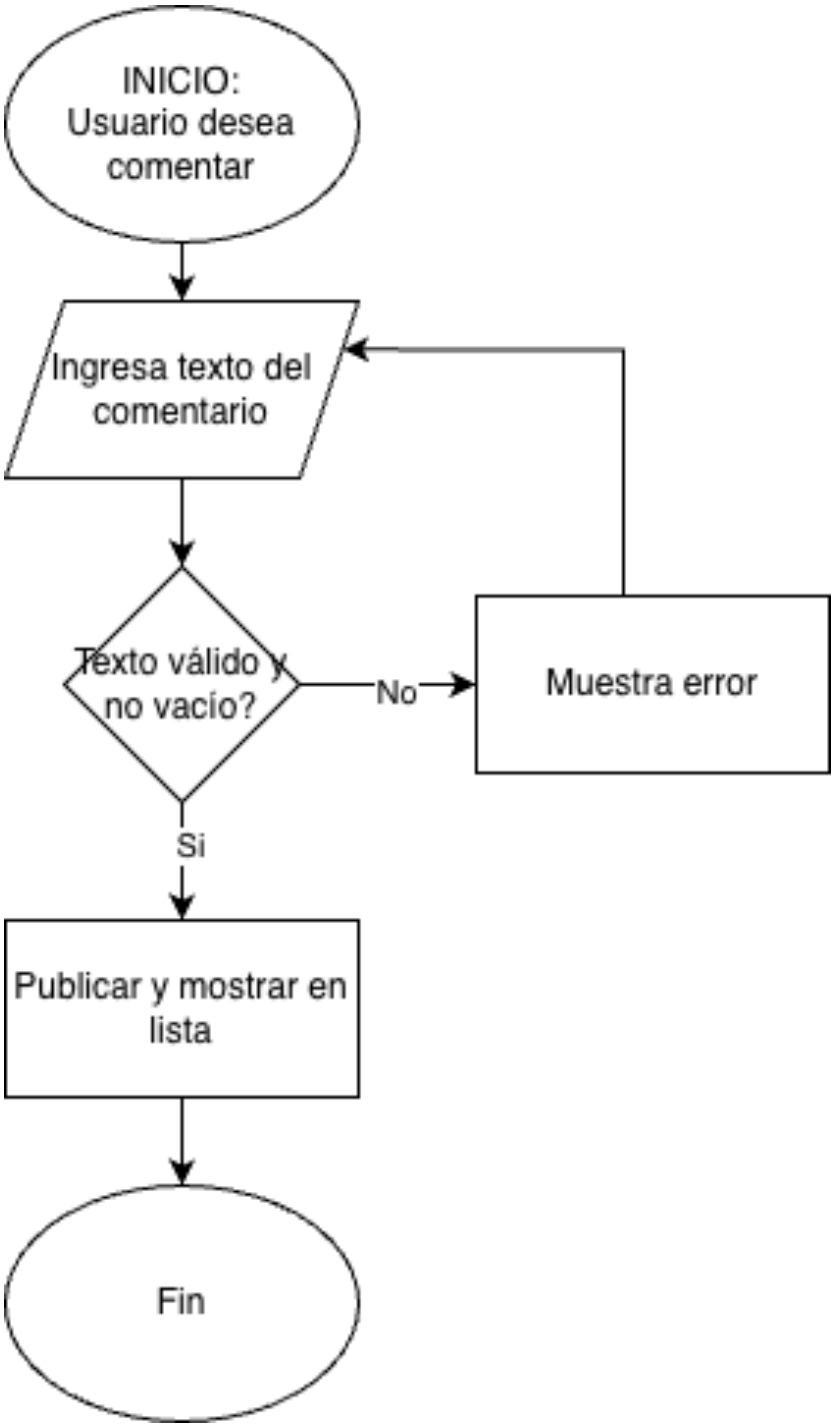
4. Diagrama de flujo, Proceso de Votación

Muestra la lógica para permitir una sola reacción por usuario, permitiendo retirar el voto o cambiarlo, actualizando los contadores de forma automática.



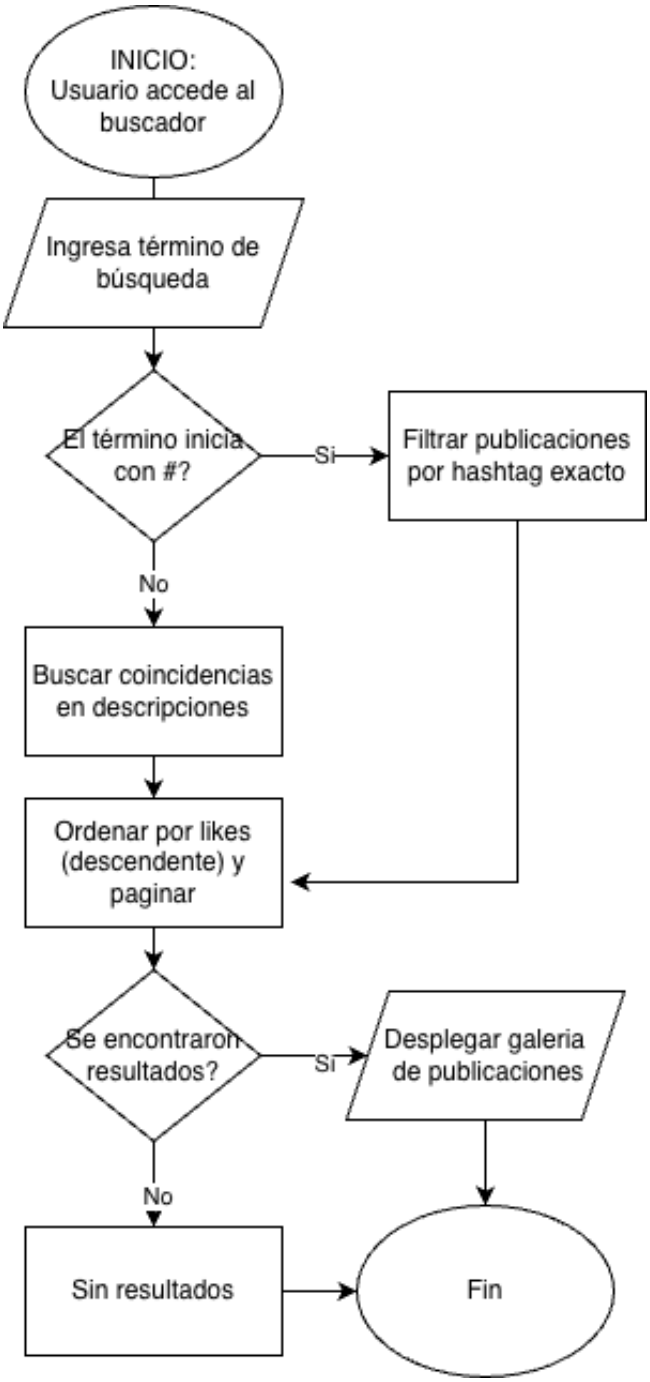
5. Diagrama de flujo, Proceso de comentarios

Este procedimiento describe la lógica de interacción social en las publicaciones. El sistema no permite el registro inmediato del comentario; primero lo somete a una validación de reglas de negocio para asegurar que el campo no este vacío y que el contenido cumpla con políticas de comunidad definidas en el sistema.



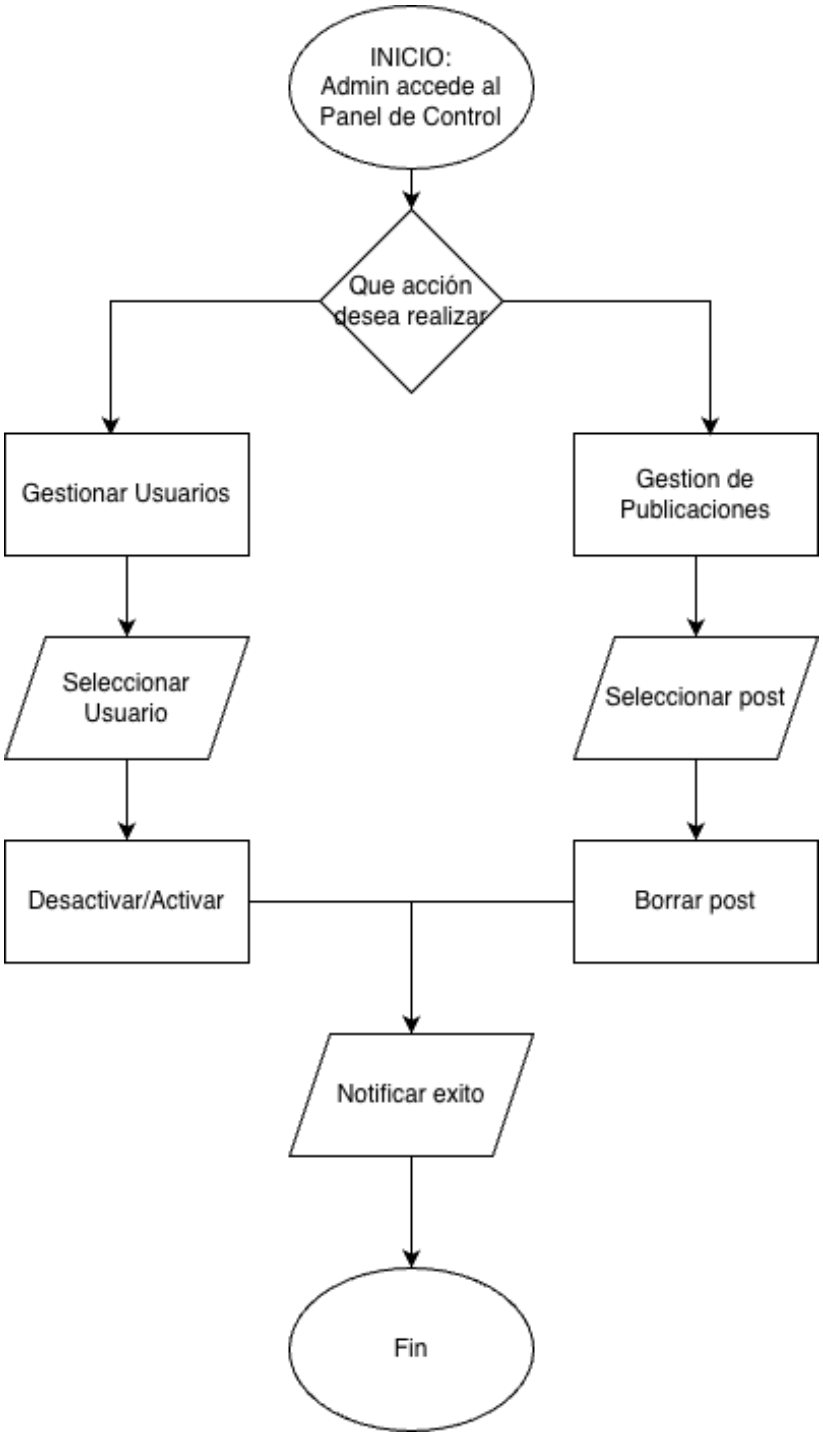
6. Diagrama de flujo, Proceso de búsqueda

Este procedimiento detalla la lógica de recuperación de información dentro de la plataforma Hipstagram. El sistema integra un motor de búsqueda que permite a los usuarios localizar contenido especifico basándose en etiquetas (hashtags) o palabras clave dentro de las descripciones de las fotografías.



7. Diagrama de flujo, Proceso de administrador

Este proceso permite al Administrador gestionar la comunidad mediante la visualización de usuarios y la eliminación de contenido que no cumpla con las normas, interactuando directamente con los microservicios de la plataforma.



Listado de Requerimientos Funcionales

- **RF-01 Gestión de sesión:** El sistema debe permitir el registro de usuarios con nombre completo, correo electrónico único y contraseña hasheada con bcrypt (mínimo 10 salt rounds). El inicio de sesión debe emitir un token JWT firmado con payload que incluye id, username y rol, con expiración de 15 minutos, y un refresh token con expiración de 7 días. El cierre de sesión invalida el token en el cliente. Todo intento de login (exitoso o fallido) debe registrarse en el log de auditoría con timestamp, IP de origen y resultado.
- **RF-02 Publicación de contenido:** El sistema debe permitir al usuario autenticado (rol USER) subir un archivo de imagen en formato JPG o PNG a través del post-service. La imagen se carga a AWS S3 mediante multer y su URL pública se persiste en PostgreSQL. La publicación debe incluir una descripción de máximo 128 caracteres (opcional) y hashtags normalizados en minúsculas. La inserción en las tablas posts y post_hashtags debe ocurrir dentro de una transacción con SAVEPOINT, de modo que un fallo en la inserción de hashtags revierta solo esa parte sin afectar el registro principal del post. Toda publicación creada debe registrarse en el log de auditoría.
- **RF-03 Gestión de contenido propio:** El sistema debe permitir al autor de una publicación eliminarla. El administrador también puede eliminar cualquier publicación. Si existen registros dependientes (votos o comentarios), el sistema debe retornar un error 409 con mensaje descriptivo.
- **RF-04 Visualización de feed:** El sistema debe exponer un endpoint de feed que retorne publicaciones con estado PUBLICADO, ordenadas por total_likes de forma descendente y fecha de creación como criterio de desempate. Los resultados deben estar paginados (parámetros page y limit, máximo 20 por página). Cada elemento del feed debe incluir: username del autor, URL de imagen firmada (S3), descripción, lista de hashtags, contador de likes, contador de dislikes y fecha de publicación. Las publicaciones sin hashtags tienen menor probabilidad de aparecer en el feed. El modo Explorar expone el top de publicaciones con más likes sin filtro adicional.
- **RF-05 Interacción de votos:** El sistema debe permitir votar con like o dislike en publicaciones de otros usuarios. Un usuario solo puede tener una reacción activa por publicación, garantizada mediante restricción UNIQUE en base de datos. Debe soportar cambio de reacción y cancelación de voto, actualizando los contadores en la misma transacción.
- **RF-06 Interacción de comentarios:** El sistema debe permitir crear comentarios de máximo 500 caracteres en cualquier publicación, listarlos paginados (10 por página) ordenados cronológicamente, y eliminarlos si el solicitante es el autor del comentario.
- **RF-07 Motor de búsqueda:** El sistema debe soportar dos modalidades de búsqueda: (a) búsqueda por hashtag exacto mediante JOIN con la tabla post_hashtags, filtrando publicaciones que contengan el hashtag indicado; (b) búsqueda por texto libre mediante ILIKE sobre la columna descripción, permitiendo coincidencias parciales. Ambas

modalidades deben retornar resultados paginados, ordenados por total_likes descendente, con los mismos campos del feed. El search-service debe operar de forma independiente sobre la base de datos relacional, sin dependencia directa del post-service en tiempo de ejecución.

- **RF-08 Perfil de usuario:** El sistema debe permitir al usuario autenticado visualizar su información de perfil y el listado de sus publicaciones.
- **RF-09 Panel de administración:** El sistema debe proveer un panel exclusivo para el rol ADMIN que permita: (a) listar, buscar, activar y bloquear cuentas de usuario; (b) listar publicaciones pendientes de moderación y cambiar su estado (PUBLICADO/BLOQUEADO/PENDIENTE); (c) gestionar la lista de palabras y hashtags prohibidos en formato JSON, con efecto inmediato sobre el microservicio de moderación; (d) consultar el log de auditoría paginado con filtros por fecha, usuario, acción y resultado. Todos los endpoints del admin-service deben validar el rol ADMIN mediante middleware antes de procesar la solicitud.
- **RF-10 Log de auditoría:** El sistema debe registrar automáticamente los eventos de login, registro, creación/eliminación de posts, votos, comentarios y acciones administrativas en una lista doblemente enlazada en memoria con persistencia en la tabla audit_logs de PostgreSQL, accesible únicamente por el rol ADMIN.

Definición del Alcance

El proyecto Hipstagram abarca el diseño, desarrollo e implementación de una plataforma de red social orientada a la publicación y descubrimiento de fotografías, accesible mediante interfaz web (Angular) y móvil. El sistema soporta dos roles diferenciados: USER, que puede publicar fotografías con descripción y hashtags, votar, comentar y buscar contenido; y ADMIN, que gestiona usuarios, modera publicaciones, administra la lista de hashtags prohibidos y consulta el log de auditoría. Toda publicación debe pasar por moderación automática de imagen y filtrado de hashtags antes de hacerse visible en el feed.

A nivel técnico, el alcance comprende la construcción de cinco microservicios independientes (auth-service, post-service, vote-service, comment-service, search-service) más un API Gateway, todos contenerizados con Docker y orquestados con Docker Compose. Incluye la implementación de autenticación JWT con bcrypt, RBAC por endpoint, integridad referencial con RESTRICT en claves foráneas de PostgreSQL, transacciones con COMMIT/ROLLBACK/SAVEPOINT, log de auditoría con lista doblemente enlazada, almacenamiento de imágenes en AWS S3, pipeline CI/CD con Jenkins y SonarQube, y pruebas unitarias, de integración y de carga con evidencias documentadas.

Como producto final se espera entregar: (1) aplicación web funcional desplegada en AWS EC2 accesible mediante HTTPS; (2) cinco microservicios operativos con sus endpoints documentados en OpenAPI; (3) base de datos PostgreSQL en AWS RDS con modelo relacional completo, restricciones FK, roles de BD y scripts GRANT/REVOKE; (4) almacenamiento de imágenes en

AWS S3 con URLs firmadas; (5) pipeline CI/CD operativo en Jenkins con Quality Gate de SonarQube aprobado; (6) log de auditoría accesible exclusivamente por el rol ADMIN con filtros por fecha, usuario, acción y resultado; (7) evidencias de pruebas unitarias por microservicio, pruebas de integración end-to-end y pruebas de carga sobre endpoints críticos; y (8) documentación técnica completa (DERCAS, arquitectura, contratos API y manual de despliegue).

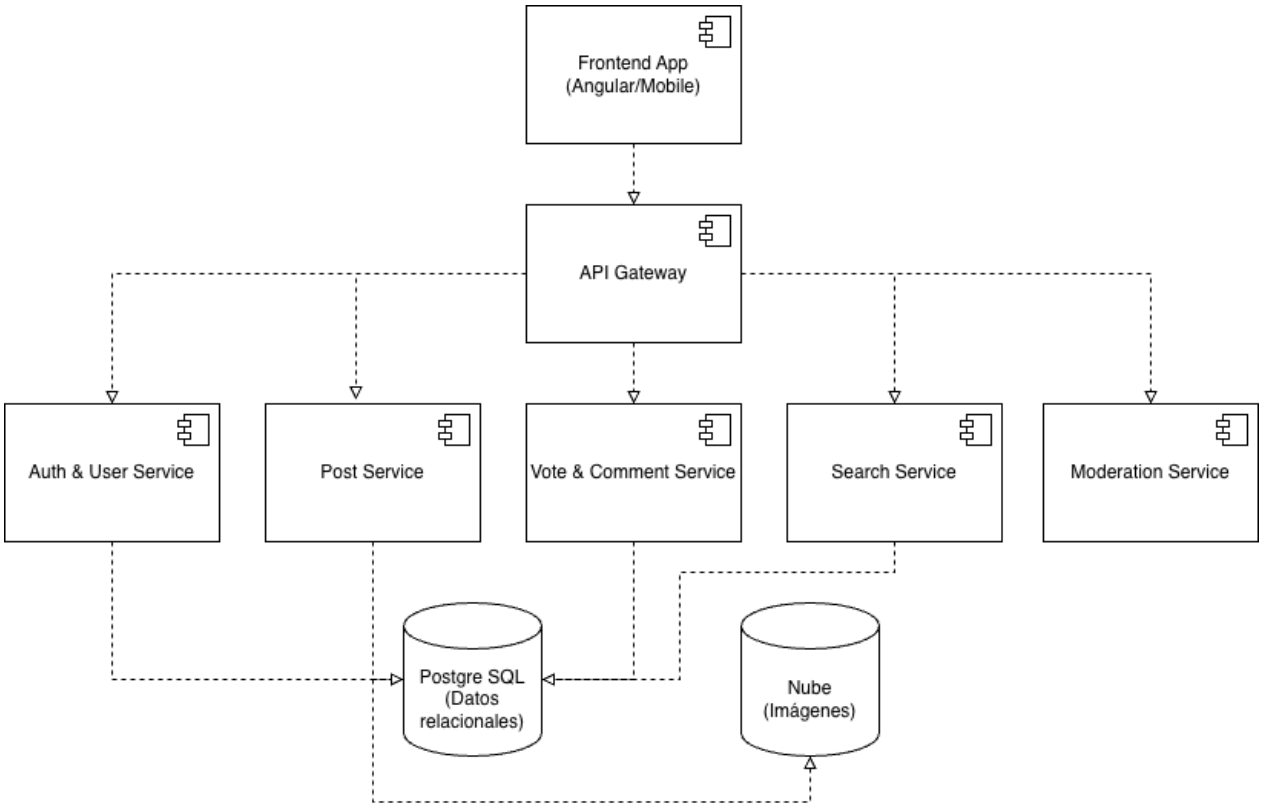
Identificación de módulos que integran la aplicación

El sistema Hipstagram se estructura en microservicios independientes, cada uno con su propia responsabilidad, base de código y contenedor Docker. Cada microservicio expone una API REST, se comunica a través del API Gateway y puede desplegarse, escalarse y actualizarse de forma autónoma. Los módulos principales son:

- **Auth Service:** Microservicio independiente en Node.js/Express que expone los endpoints /registro y /login. Gestiona el hash de contraseñas con bcrypt, emite tokens JWT firmados con rol incluido, y valida credenciales contra PostgreSQL. Opera de forma autónoma sin dependencia de otros microservicios.
- **Post Service:** Microservicio que gestiona la creación, consulta y eliminación de posts. Recibe imágenes multipart, las almacena en AWS S3 mediante multer, persiste la URL y metadatos en PostgreSQL dentro de transacciones con SAVEPOINT para la inserción de hashtags, y expone el feed ordenado por likes.
- **Vote & Comment Service:** Microservicio que procesa likes/dislikes con control de duplicidad mediante restricción UNIQUE en base de datos, y gestiona comentarios con validación de longitud y paginación. Cada operación genera un evento en el AuditLogService.
- **Search Service:** Microservicio que expone dos endpoints: búsqueda por hashtag exacto mediante JOIN con la tabla post_hashtags, y búsqueda por texto libre mediante ILIKE sobre la columna descripción. Resultados ordenados por total_likes descendente.
- **Admin Service:** Microservicio protegido por middleware de verificación de rol ADMIN. Expone endpoints para gestión de usuarios (activar/bloquear), gestión de posts (cambiar estado), y consulta del log de auditoría paginado con filtros por acción, usuario y resultado.

Diagrama de componentes

A continuación, se presenta el modelo estructural del sistema Hipstagram basado en la notación UML. Este diagrama ilustra la organización física de los módulos de software previamente identificados, mostrando las interfaces de comunicación y las relaciones de dependencia entre la aplicación cliente, el API Gateway, el ecosistema interno de microservicios y la capa de persistencia de datos y almacenamiento en la nube.



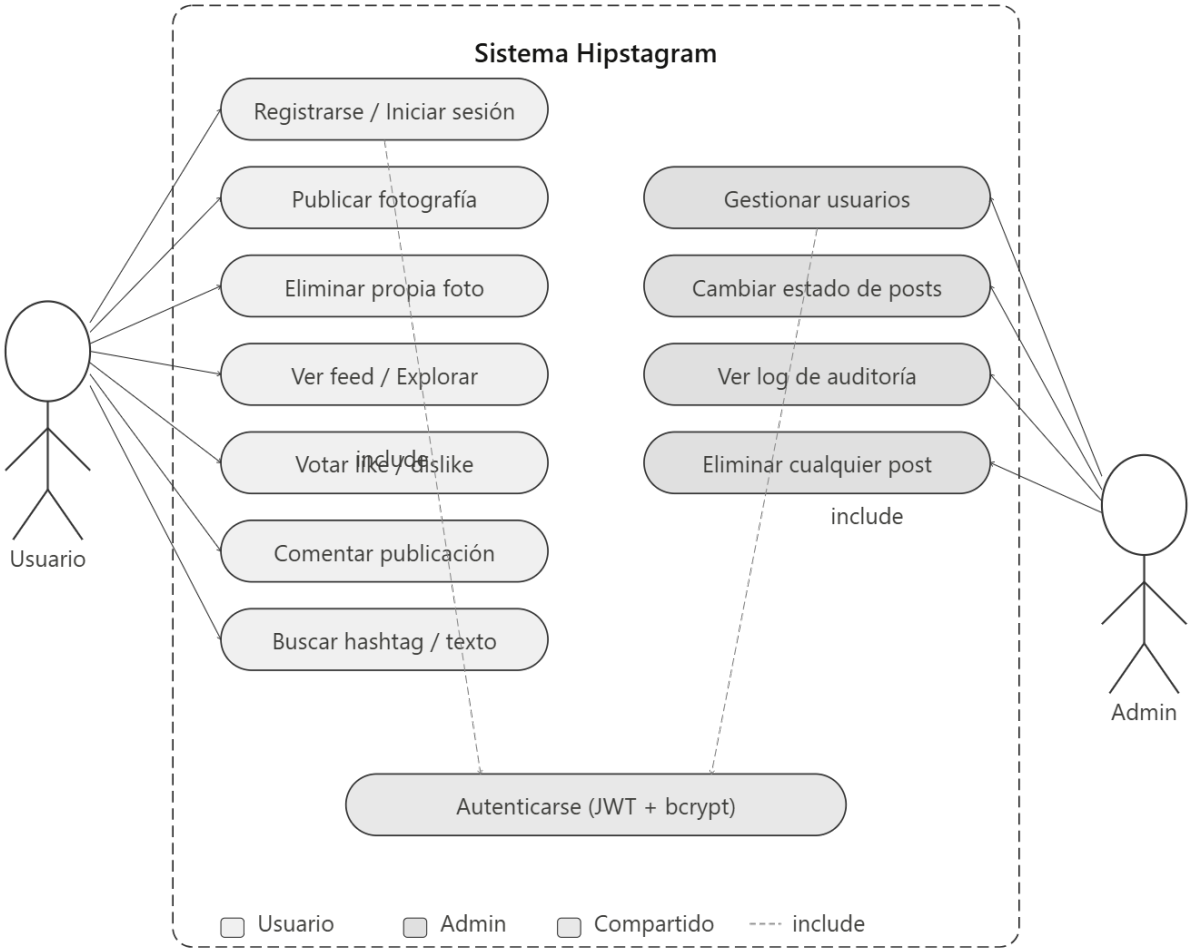
Definición de requerimientos no funcionales

Los requerimientos no funcionales definen atributos de calidad, rendimiento y seguridad que el sistema Hipstagram debe cumplir para garantizar una operación robusta y confiable.

- RNF-01 Seguridad: El sistema debe implementar hashing seguro de contraseñas (utilizando algoritmos de seguridad). La autenticación debe manejarse mediante tokens JWT con expiración corta y refresh tokens, aplicando control de acceso basado en roles.
- RNF-02 Validación de datos: Se debe aplicar una validación estricta de entradas mediante schemas, tanto a nivel del API Gateway como internamente en cada microservicio.
- RNF-03 Disponibilidad: La arquitectura debe soportar el reinicio automático de contenedores en caso de fallos y exponer endpoints de health checks para el monitoreo.
- RNF-04 Rendimiento: Las respuestas de listados (feed, resultados de búsqueda y comentarios) deben estar paginadas, y el procesamiento de votos debe ser eficiente para soportar alta concurrencia.
- RNF-05 Calidad de Código: El desarrollo debe estar respaldado por pruebas automatizadas (unitarias, de integración y de estrés), y el código debe superar las métricas de calidad establecidas en SonarQube.

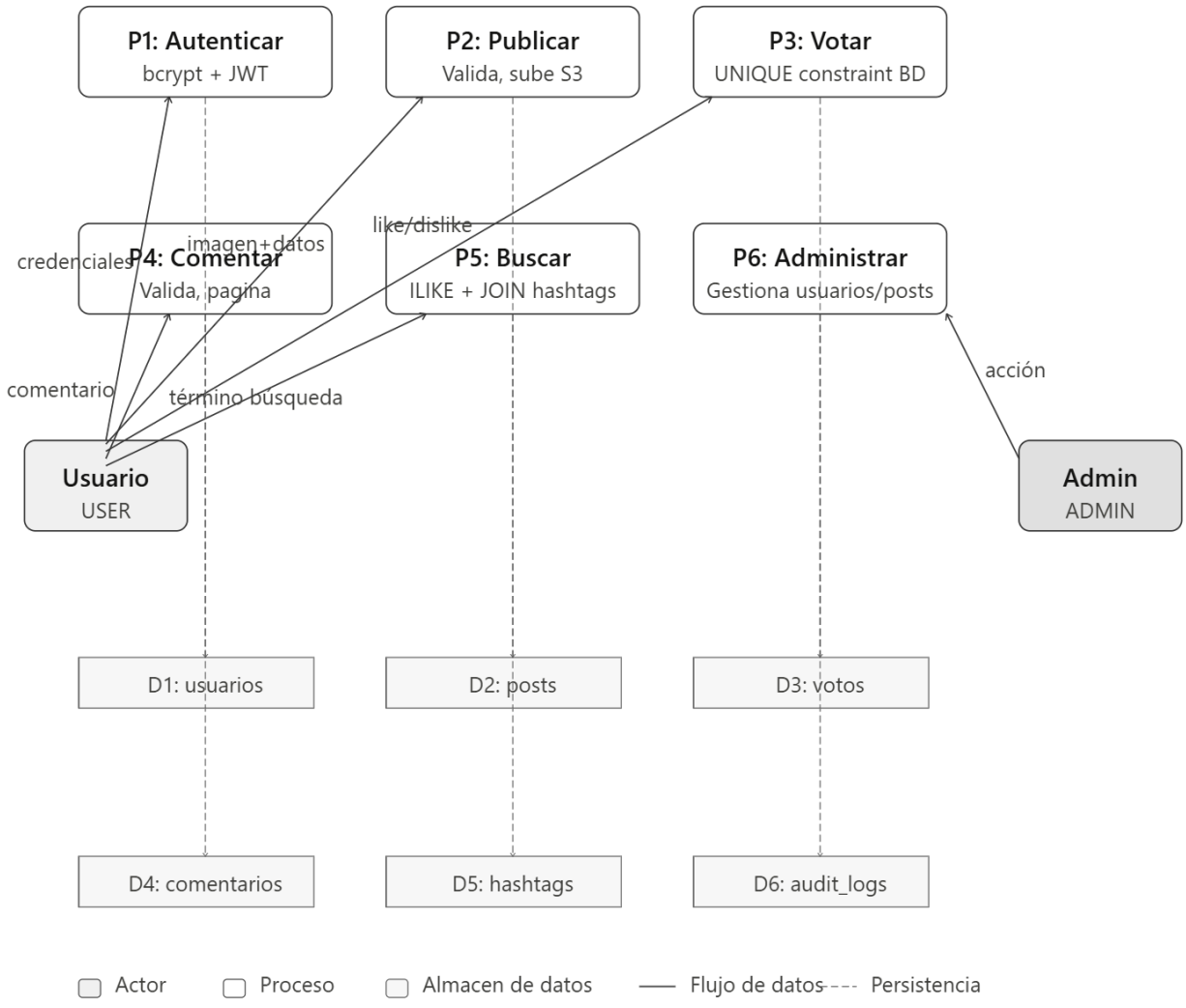
Diagramas de casos de uso

El siguiente diagrama de casos de uso modela las interacciones principales entre los actores del sistema (Usuario y Administrador) y las funcionalidades que ofrece la plataforma Hipstagram. Este modelo define los límites del sistema y grafica gráficamente el alcance funcional detallado en los requerimientos del proyecto.



Diagramas de flujo de datos

El siguiente Diagrama de Flujo de Datos, ilustra el flujo de información entre el sistema Hipstagram, sus entidades externas (Usuario y Administrador) y los procesos o almacenes de datos claves. Se puede observar cómo el Usuario interactúa con los procesos de Gestión de Publicaciones y Búsqueda, mientras que el Administrador lo hace con los de Gestión de Contenido y Gestión de Usuarios, alimentando la Base de Datos.



Prototipos e interfaces de los requerimientos funcionales

Figura 1 y 2: Pantalla de Login y Registro

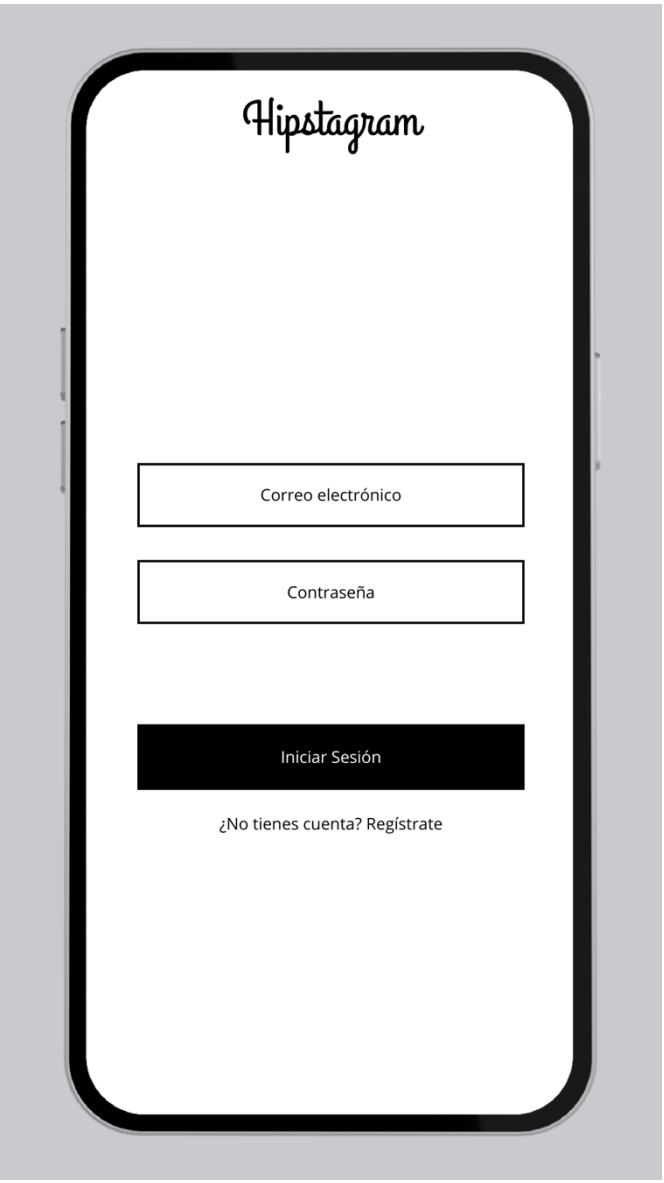


Figura 3: Feed Principal

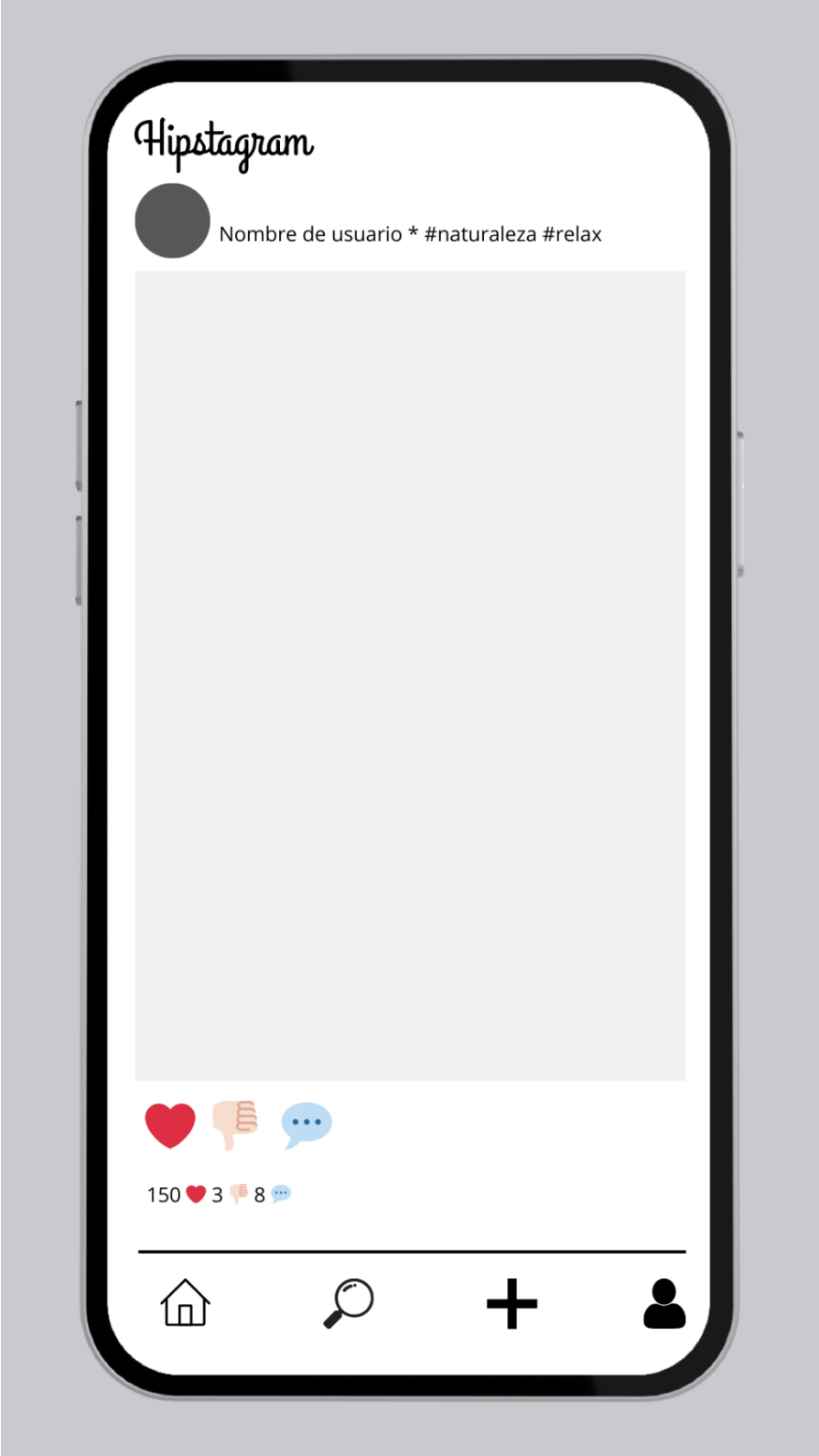


Figura 4: Detalle de publicación y comentarios

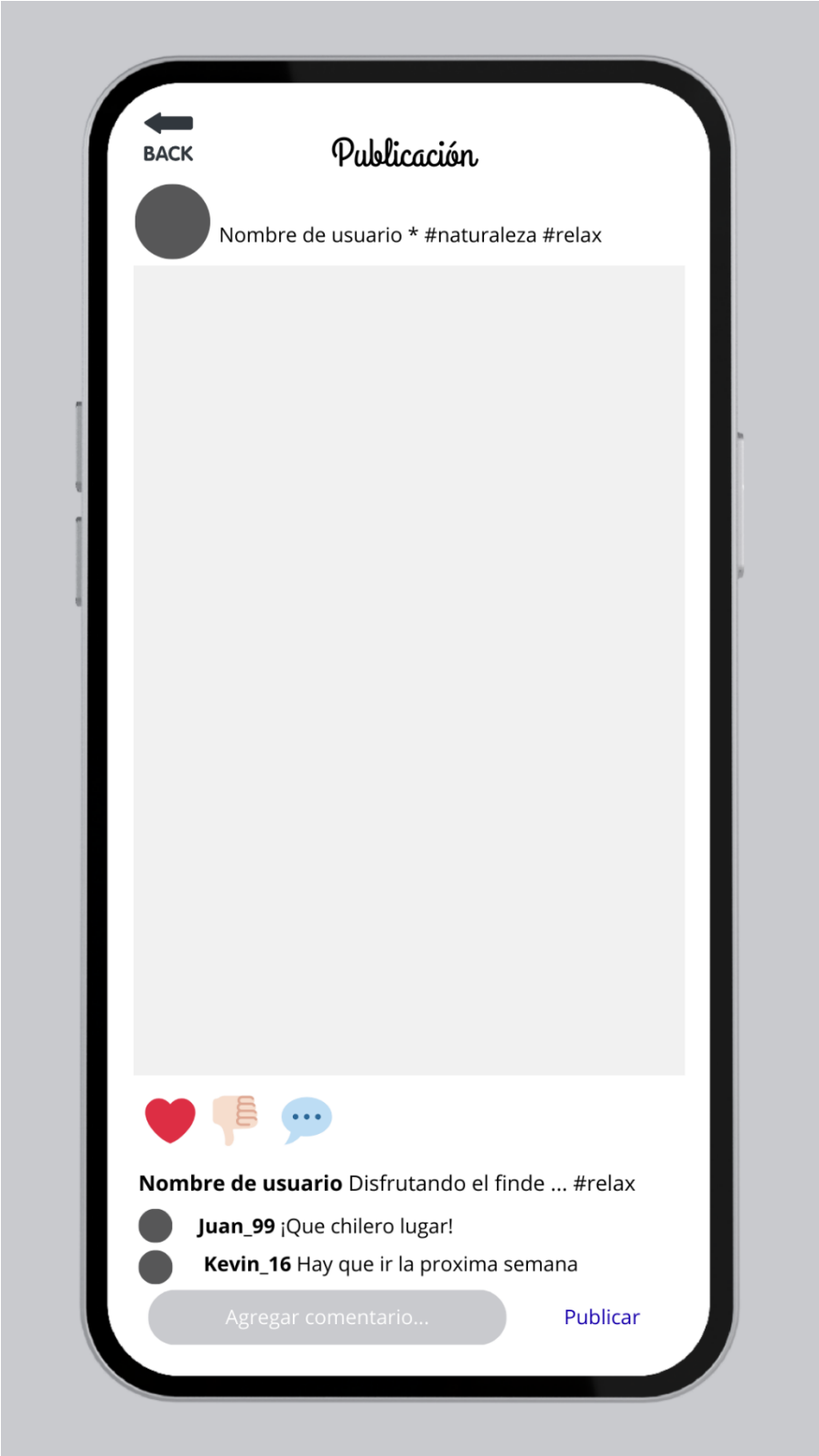


Figura 5: Búsqueda / Explorar

Figura 6: Nueva Publicación

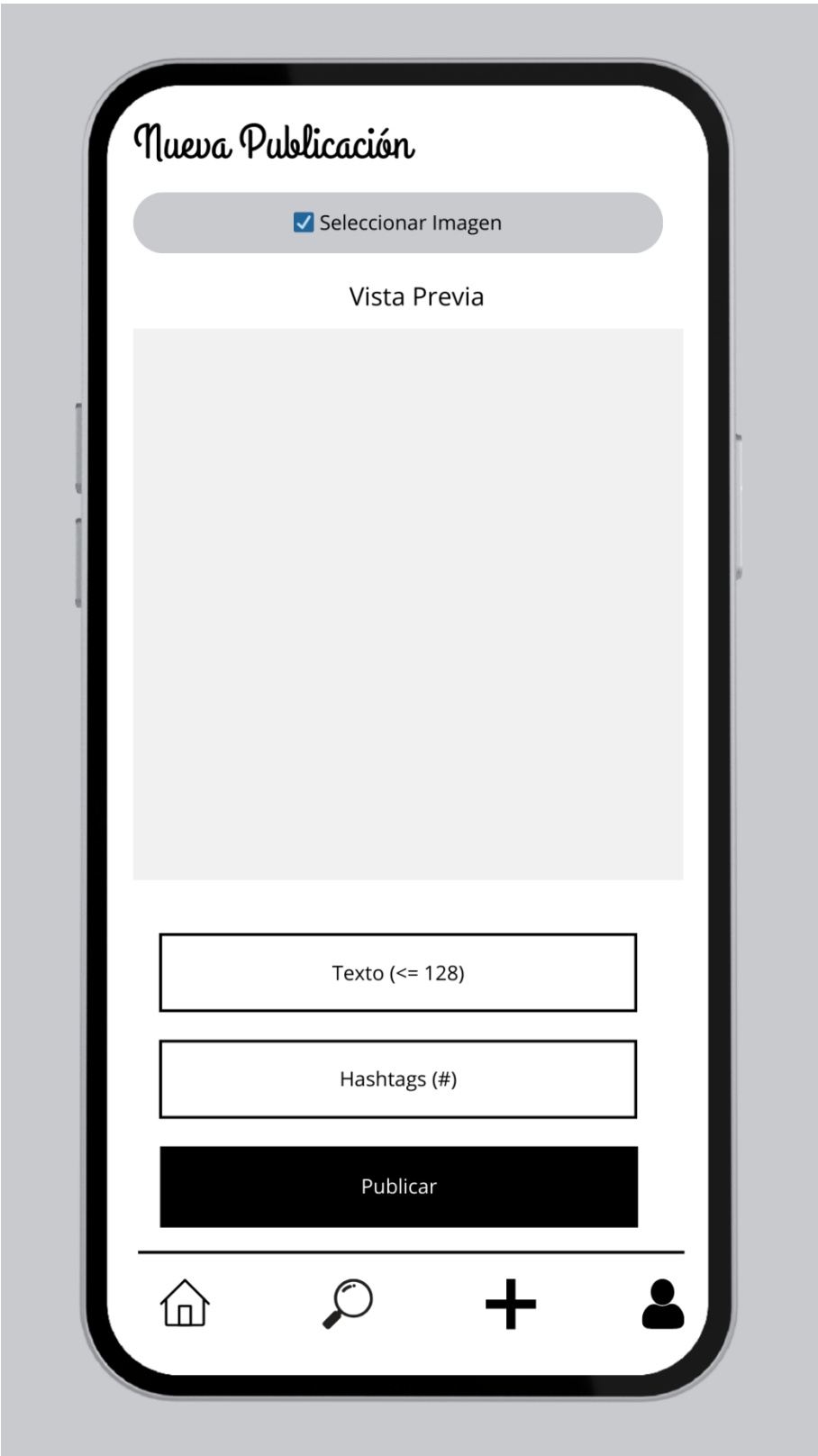


Figura 7: Panel de Administración

Recursos tecnológicos que se van a utilizar

Para el desarrollo, implementación y despliegue del proyecto Hipstagram, se utilizará una combinación de herramientas de código abierto e infraestructura de nube AWS. El presupuesto se presenta en dos fases: (1) desarrollo y pruebas, aprovechando la capa gratuita de 12 meses de AWS; (2) producción e implementación real, con los costos correspondientes a instancias y servicios fuera de la capa gratuita, incluyendo AWS RDS PostgreSQL, EC2 y S3. A continuación, se detallan los recursos y su costo mensual estimado en dólares estadounidenses.

- Control de versiones (Git y GitHub): Se utilizará para el alojamiento del repositorio de código y la aplicación de la metodología GitFlow. Su costo es de $0 (gratuito).
- Documentación de API (Open API): Herramienta para la definición y documentación técnica de los contratos de la API. Su costo de licencia es de $0 (Open Source).
- Contenedorización (Docker y Docker Compose): Tecnología empleada para el empaquetado de los microservicios, asegurando consistencia entre los entornos locales y la nube. Su costo es de $0 (Open Source).
- Integración Continua y Calidad de Código (Jenkins y SonarQube): Herramientas utilizadas para la automatización de pruebas, análisis de código estático y despliegue. Su costo es de $0 (Open Source).
- Enrutamiento en la nube (AWS API Gateway): Servicio gestionado para el enrutamiento seguro de peticiones desde el cliente hacia los microservicios. Se proyecta utilizar la capa gratuita ($0).
- Computo de la nube (AWS EC2 / ECS): Instancias (tipo t2.micro) para el alojamiento del servidor Jenkins, SonarQube y los contenedores del backend. Se estima un costo de aproximadamente $15 mensuales por posibles horas excedentes de la capa gratuita.
- Base de datos (AWS RDS con PostreSQL): Servicio gestionado de base de datos relacional PostgreSQL para la persistencia de datos de usuarios, publicaciones, votos, comentarios y audit_logs. Durante desarrollo se utiliza la instancia db.t3.micro de la capa gratuita ($0). En producción e implementación real se requiere una instancia db.t3.micro con almacenamiento de 20GB, estimada en ~$25/mes, considerando el costo de instancia, almacenamiento y transferencia de datos.
- Almacenamiento de objetos (AWS S3): Espacio destinado al almacenamiento de las fotografías subidas por los usuarios, que serán procesadas en diferentes tamaños para optimizar su visualización. Se utilizará el límite base de 5GB de la capa gratuita ($0).

- Dominio de red (varios): Recurso opcional, pero recomendado, para asegurar los endpoints de la API mediante certificados HTTPS. Representa un pago único anual estimado de $12.

Recurso	Desarrollo (Mensual)	Producción (mensual)
AWS EC2 t2.micro	$0 (capa gratuita 12 meses)	~$8/mes
AWS RDS PostgreSQL t3.micro	$0 (capa gratuita 12 meses)	~$25/mes
AWS S3 (primeros 5GB)	$0	~$0.12/GB adicional
AWS CloudWatch logs	$0	~$0.50/GB
GitHub repositorio	$0	$0
Docker / Docker Compose	$0	$0
Jenkins self-hosted en EC2	$0	$0
SonarQube Community	$0	$0
Dominio + SSL	-	~$12/año
Total estimado	$0	~$35–50/mes

Prioridad de los requerimientos a desarrollar

Con base en el plan de trabajo estructurado en iteraciones (sprints) y la dependencia técnica entre los diferentes módulos de la arquitectura del software, se ha establecido la siguiente priorización para el desarrollo y entrega de los requerimientos funcionales:

Prioridad alta (Semanas 5 a 8)

Estos requerimientos conforman el núcleo (core) del sistema y son absolutamente indispensables para el funcionamiento básico de la plataforma. Su desarrollo debe completarse en las primeras iteraciones:

- RF-01 Gestión de Sesión: Es la base de la seguridad y permite el acceso controlado al sistema.
- RF-02 Publicación de Contenido: Representa la función principal de la plataforma (subida de fotografías).
- RF-03 Gestión de Contenido Propio: Es un requisito fundamental de control de datos para que el usuario gestione sus fotografías.
- RF-08 Perfil de Usuario: Es estrictamente necesario para vincular las fotografías y acciones de un autor especifico en la base de datos.

Prioridad Media (Semanas 9 a 13)

Estas funcionalidades añaden la capa de interacción social y descubrimiento a la red social. Su desarrollo depende de que la estructura de la prioridad alta ya este construida y estable:

- RF-04 Visualización de Feed: Requiere que ya exista contenido y un algoritmo de conteo de likes estructurado.
- RF-05 Interacción de votos: Depende de la existencia previa de publicaciones para registrar las reacciones.
- RF-06 Interacción de comentarios: Depende de la estructura de publicaciones para poder enlazar las respuestas de los usuarios.
- RF-07 Motor de Búsqueda: Representa la funcionalidad de descubrimiento secundario para navegar por el contenido mediante texto y hashtags.
- RF-09 Acceso Administrativo: Involucra la creación del panel base y el control de roles para iniciar la gestión operativa.

Prioridad Baja (Semanas 14 a 16)

Agrupan las características de administración avanzada las cuales se implementarán como última fase, tras haber probado y estabilizado el flujo principal del usuario final:

- RF-07 Optimización de Búsqueda: Refinamiento de los filtros de texto para mejorar la precisión de los resultados en el feed "Explorar".
- Gestión Administrativa: Implementación de la interfaz para que el administrador pueda dar de baja usuarios que incumplan las normas.
- QA y Pruebas de Carga: Ejecución de pruebas de rendimiento en los microservicios de interacciones para asegurar estabilidad bajo alta concurrencia.